Comprehensive Explanation of 5-Robot Pentagon Formation NMPC System

Code Analysis and Documentation

December 21, 2025

1 Overview and Imports

The code implements a 5-robot system that cooperatively carries a pentagonal object while navigating through static and dynamic obstacles. The robots grasp the object at the midpoints of the pentagon's edges, maintaining a rigid formation throughout the motion. The system uses CasADi for non-linear optimization, NumPy for numerical computations, and Matplotlib for visualization and animation. This represents a sophisticated multi-agent control problem combining trajectory tracking, formation maintenance, manipulation, and obstacle avoidance.

2 SAT Collision Detection

The `sat_gap_fixed` function (Lines 18–47) implements the Separating Axis Theorem (SAT) to detect collisions between polygons. It works by projecting both polygons onto all potential separating axes—the normals to each polygon's edges. For each axis, it computes the minimum and maximum projection values for both polygons. If there's a gap between the projections on any axis, the polygons don't collide. The function returns the minimum gap found across all axes—a positive value indicates separation (no collision), while a negative value indicates overlap (collision).

The `create_hexagon_symbolic` function generates hexagonal obstacle representations using CasADi expressions, enabling symbolic differentiation necessary for optimization. The `create_hexagon_numpy` function creates the same hexagonal geometry using NumPy arrays for visualization purposes. This dual approach separates the symbolic computation needed for optimization from the numerical computation needed for graphics.

3 Triangle Point Generation

The `calculate_triangle_points_sym` function (Lines 50–63) computes three key points that form a triangle representing a robot's footprint. Given a robot base position $\mathbf{R}$, end-effector position $\mathbf{A}$, and robot radius r, it calculates:

- Point $\mathbf{A}$: the end-effector itself

- Points **B** and **C**: vertices of a tangent triangle

This triangle ensures the robot's circular body doesn't collide with the grasp point. The function uses the arc sine to find the tangency angle:

$$\theta = \arcsin\left(\min\left(\frac{r}{\|\mathbf{A}-\mathbf{R}\|}, 0.999\right)\right)$$

Then it projects this to find the tangent line from the base to the end-effector, with points **B** and **C** positioned perpendicular to this line.

4 Angular Non-Collinearity Constraint

The `compute_proper_angular_constraint` function (Lines 66–93) ensures that the robot's triangular footprint doesn't become collinear with the pentagon edges it's grasping. For a valid grasp, the robot's body points (**B** and **C**, which define the tangent circle) must maintain a minimum angle relative to the pentagon's edge directions.

The function computes unit vectors along the pentagon edges and the robot's body, then calculates the dot product to find the angle between them:

$$h_{\text{AB-left}} = \cos(\theta_{\min}) - |\cos(\angle(\mathbf{u}_{AB}, \mathbf{u}_{\text{edge-left}}))|$$

It returns a barrier function value that becomes negative if the constraint is violated (collinearity becomes too acute). This constraint is critical for maintaining grasp validity throughout the motion.

5 Dynamic Obstacle Trajectory Generation

Three helper functions (Lines 96–114) generate obstacle motion patterns:

- `generate_circular_trajectory`: Creates a circular path around a center point with specified angular velocity ω:

$$\mathbf{p}_{\text{obs}}(t) = \mathbf{c} + r\begin{pmatrix}\cos(\omega t)\\ \sin(\omega t)\end{pmatrix}$$

- `generate_linear_trajectory`: Creates straight-line motion with constant velocity $\mathbf{v}$:

$$\mathbf{p}_{\text{obs}}(t) = \mathbf{p}_0 + \mathbf{v}\cdot t$$

- `get_velocity_from_trajectory`: Computes obstacle velocity at any timestep using finite differencing:

$$\mathbf{v}(k) = \frac{\mathbf{p}(k+1) - \mathbf{p}(k)}{\Delta t}$$

This velocity computation is crucial for the velocity-based CBF constraints that need to know obstacle motion for predictive avoidance.

6 Pentagon Geometry Setup

The `create_pentagon_vertices` function (Lines 117-128) generates the five vertices of a regular pentagon given its center and circumradius (distance from center to vertex):

$$\mathbf{v}_i = \mathbf{c} + r_{\text{circum}} \begin{pmatrix} \cos\left(\frac{2\pi i}{5} - \frac{\pi}{2}\right) \\ \sin\left(\frac{2\pi i}{5} - \frac{\pi}{2}\right) \end{pmatrix}$$

Starting from the top for consistent orientation. The `get_pentagon_midpoints` function computes the midpoints between consecutive vertices:

$$\mathbf{m}_i = \frac{\mathbf{v}_i + \mathbf{v}_{i+1}}{2}$$

These midpoints serve as the five grasp points where the robots hold the pentagon, transforming the pentagon geometry into the manipulation interface.

7 Parameter Definition

The code defines fundamental system parameters (Lines 131–185). Key parameters include:

- **Time Step**: $\Delta t = 0.1$ seconds
- **Horizon Length**: $N = 20$ steps (2-second planning window)
- **Number of Robots**: $n_r = 3$
- **Robot State**: $\mathbf{x}_i = [x_b, y_b, q_1, d_2, q_3]^T$ (base position and three arm joints)
- **Robot Control**: $\mathbf{u}_i = [v_x, v_y, \dot{q}_1, \dot{d}_2, \dot{q}_3]^T$
- **Arm Link Lengths**: $L_1 = L_3 = 0.05$ m (fixed), $d_2 \in [0.1, 0.5]$ m (variable)

The cost matrices define optimization priorities. The object tracking cost matrix $\mathbf{Q}_{\text{obj}} = \text{diag}(150, 150)$ defines how strongly the system tracks the reference trajectory—higher values mean stricter tracking. The control cost matrix $\mathbf{R}$ penalizes control effort, with higher penalties on arm movements ($R_{43} = 50$) to keep the arms stable and energy-efficient.

7.1 Safety Parameters

The Control Barrier Function (CBF) parameters define safety distances:

- **Inter-robot Clearance**: $d_{\text{min,inter}} = 0.20$ m
- **Base to Obstacle**: $d_{\text{min,base-obs}} = 0.50$ m
- **Object to Obstacle**: $d_{\text{min,obj-obs}} = 0.40$ m
- **CBF Gain**: $\gamma_{\text{CBF}} = 10.0$ (larger gains ensure faster constraint satisfaction)
- **Maximum Slack Allowed**: $\varepsilon_{\max} = 0.01$ (small slack for near-violations in tight maneuvers)

The rigidity constraint weight $W_{\text{rigidity}} = 10^7$ is extremely high, making formation deformation heavily penalized relative to other objectives.

8 Initial Condition Setup

The initialization (Lines 187–234) establishes valid starting configurations for all robots. The pentagon object starts at position $(1.5, 2.5)$ m. For each robot i:

1. The grasp point $\mathbf{m}_i$ is located at the midpoint of pentagon edge i
2. The base position is calculated as:

$$\mathbf{b}_i = \mathbf{m}_i + \frac{\mathbf{m}_i - \mathbf{c}_{\text{obj}}}{\|\mathbf{m}_i - \mathbf{c}_{\text{obj}}\|} \cdot L_{\text{arm}}$$

   where L_{arm} is the initial arm length (sum of all segments)

3. The arm angles are computed to match this configuration:

$$q_1 = \arctan 2(m_{i,y} - b_{i,y}, m_{i,x} - b_{i,x})$$

   with d_2 initialized to 0.3 m and q_3 to 0

This ensures all robots start in a valid configuration with their end-effectors at the pentagon grasp points.

9 Forward Kinematics

The `fk_ee_world_np` function (Lines 237–252) computes the end-effector position using NumPy:

$$\begin{pmatrix} x_e \\ y_e \end{pmatrix} = \begin{pmatrix} x_b \\ y_b \end{pmatrix} + (L_1 + d_2) \begin{pmatrix} \cos(q_1) \\ \sin(q_1) \end{pmatrix} + L_3 \begin{pmatrix} \cos(q_1 + q_3) \\ \sin(q_1 + q_3) \end{pmatrix}$$

where:

- (x_b, y_b) is the robot base position
- L_1, L_3 are fixed link lengths
- d_2 is the variable-length middle segment
- q_1, q_3 are joint angles

The `fk_ee_sym` function implements the same computation using CasADi symbolic expressions. This symbolic version is crucial because the optimizer must differentiate through these expressions to compute gradients and Jacobians for the optimization problem.

10 Reference Trajectory Generation

The code generates two types of reference trajectories (Lines 261–277) for the object's centroid to follow:

- **Straight-line trajectory**:

$$\mathbf{c}_{\text{ref}}(k) = \mathbf{c}_0 + k \cdot \Delta t \cdot \frac{\mathbf{c}_{\text{goal}} - \mathbf{c}_0}{T}$$

- **Sinusoidal trajectory** (selected):

$$\mathbf{c}_{\text{ref}}(k) = \begin{pmatrix} c_{0,x} + 0.5 \cdot k \cdot \Delta t \\ c_{0,y} + A \sin(\omega \cdot k \cdot \Delta t) \end{pmatrix}$$

  with amplitude $A = 1.0$ m and frequency $\omega = 0.2$ rad/s

The sinusoidal reference tests whether the formation can execute evasive maneuvers while avoiding obstacles and maintaining formation integrity.

11 Obstacle Configuration

The system includes static and dynamic obstacles (Lines 280–302):

11.1 Static Obstacles

Three hexagon-shaped obstacles with radius $r_{\text{static}} = 0.35$ m positioned at fixed locations:

- Obstacle 1: center at $(4.0, 3.5)$ m
- Obstacle 2: center at $(12.0, 1.5)$ m
- Obstacle 3: center at $(7.0, 7.5)$ m

11.2 Dynamic Obstacles

Two moving obstacles test real-time avoidance:

- **Obstacle 1 (Circular Motion)**: Orbits around center $(8.0, 1.5)$ m with orbital radius 0.8 m and angular velocity $\omega = 0.3$ rad/s
- **Obstacle 2 (Oscillation)**: Oscillates along the x-axis from base position $(7.5, 4.2)$ m with sinusoidal motion:

$$\mathbf{p}_{\text{obs},2}(t) = \begin{pmatrix} 7.5 + 2.0 \sin(0.2t) \\ 4.2 \end{pmatrix}$$

Dynamic obstacle radius: $r_{\text{dyn}} = 0.30$ m. These obstacles are strategically positioned to create bottlenecks that test the formation's navigation capability.

12 Optimization Problem Formulation

The CasADi Opti object (Lines 305–330) sets up a non-linear optimization problem:

- **Decision Variables**:
  - $\mathbf{X} \in \mathbb{R}^{(n_r \cdot 5)\times(N+1)}$: state trajectory
  - $\mathbf{U} \in \mathbb{R}^{(n_r \cdot 5)\times N}$: control inputs
  - $s_{\text{cbf}} \in \mathbb{R}^{n_{\text{cbf}}\times N}$: CBF constraint slack variables
  - $s_{\text{rig}} \in \mathbb{R}^{n_{\text{rig}}\times N}$: rigidity constraint slack variables
- **Parameters**:
  - $\mathbf{X}_{\text{ref}} \in \mathbb{R}^{2\times(N+1)}$: reference object trajectory
  - $\mathbf{x}_0 \in \mathbb{R}^{n_r \cdot 5}$: initial state
  - $\{\mathbf{c}_{\text{obs}}\}$: static obstacle centers
  - $\{\mathbf{c}_{\text{dyn}}, \mathbf{v}_{\text{dyn}}\}$: dynamic obstacle positions and velocities

The slack variables allow constraint violations when necessary, heavily penalized in the cost function to enforce safety while maintaining feasibility during tight maneuvers.

13 Dynamics and State Bounds

The system dynamics (Lines 332–348) are purely kinematic:

$$\mathbf{b}_{i,k+1} = \mathbf{b}_{i,k} + \Delta t \cdot \mathbf{u}_{i,k}^{\text{base}}$$

$$\mathbf{a}_{i,k+1} = \mathbf{a}_{i,k} + \Delta t \cdot \mathbf{u}_{i,k}^{\text{arm}}$$

where $\mathbf{b}_{i,k}$ is the base position and $\mathbf{a}_{i,k} = [q_1, d_2, q_3]^T$ is the arm configuration. State bounds constrain the workspace:

$$\begin{cases} x_b, y_b \in [-20, 20] \text{ m} & \text{(base position)} \\ q_1, q_3 \in [-\pi, \pi] \text{ rad} & \text{(joint angles)} \\ d_2 \in [0.1, 0.5] \text{ m} & \text{(middle link length)} \end{cases}$$

These bounds prevent the arm from reaching unphysical configurations and keep the system within the operational workspace.

14 Cost Function Design

The cost function (Lines 351–400) combines multiple objectives weighted by their importance:

$$J = J_{\text{track}} + J_{\text{control}} + J_{\text{form}} + J_{\text{prog}} + J_{\text{slack}} \tag{1}$$

14.1 Tracking Cost

$$J_{\text{track}} = \sum_{k=0}^{N-1} \|\mathbf{c}_k - \mathbf{x}_{\text{ref}}(k)\|^2_{\mathbf{Q}_{\text{obj}}} + \|\mathbf{c}_N - \mathbf{x}_{\text{ref}}(N)\|^2_{\mathbf{Q}_{\text{terminal}}}$$

where $\mathbf{c}_k = \frac{1}{n_r}\sum_{i=1}^{n_r} \mathbf{e}_{i,k}$ is the object centroid (mean of end-effector positions). Terminal cost $\mathbf{Q}_{\text{terminal}} = 600 \cdot \mathbf{I}$ is much higher to ensure reaching the goal.

14.2 Control Cost

$$J_{\text{control}} = \sum_{k=0}^{N-1} \mathbf{u}_k^T \mathbf{R} \mathbf{u}_k$$

where $\mathbf{R} = \text{diag}(\text{tile}([1, 1, 10, 50, 10], n_r))$ penalizes large arm movements more heavily.

14.3 Formation Keeping Cost

$$J_{\text{form}} = Q_{\text{form}} \sum_{k=0}^{N-1} \sum_{i=1}^{n_r} \|\mathbf{b}_{i,k} - (\mathbf{c}_k + \Delta\mathbf{b}_i^0)\|^2$$

where $\Delta\mathbf{b}_i^0 = \mathbf{b}_{i,0} - \mathbf{c}_0$ is the initial offset of robot base from object centroid. This penalizes deviations of robot bases from their expected positions relative to the centroid.

14.4 Progress Incentive

$$J_{\text{prog}} = -k_{\text{prog}} \sum_{k=0}^{N-1} \bar{v}_{x,k}$$

where $\bar{v}_{x,k} = \frac{1}{n_r}\sum_{i=1}^{n_r} u_{i,k}^{v_x}$ is the average base velocity in the x-direction. This negative term rewards forward progress.

14.5 Slack Variable Penalties

$$J_{\text{slack}} = w_{\text{cbf}}\|s_{\text{cbf}}\|_F^2 + w_{\text{rig}}\|s_{\text{rig}}\|_F^2$$

with extremely high weights ($w_{\text{cbf}} = 5 \times 10^5$, $w_{\text{rig}} = 10^7$) to enforce safety and formation integrity.

15 Velocity-Based CBF Constraints

The code implements velocity-based Control Barrier Functions for six types of safety constraints (Lines 403–529). The general form of a velocity-based CBF is:

$$\dot{h} + \gamma h \geq -\varepsilon$$

where h is the barrier function (positive when safe), $\dot{h}$ is its time derivative, $\gamma > 0$ is the CBF gain, and $\varepsilon \geq 0$ is slack.

15.1 Inter-Robot Collision Avoidance

$$h_{\text{inter},ij} = \|\mathbf{b}_i - \mathbf{b}_j\|^2 - d^2_{\text{min,inter}}$$
$$\dot{h}_{\text{inter},ij} = 2(\mathbf{b}_i - \mathbf{b}_j)^T(\mathbf{v}_i - \mathbf{v}_j)$$

A "gating" mechanism deactivates this constraint when robots are far apart ($\|\mathbf{b}_i - \mathbf{b}_j\| > d_{\text{gate}}$) to reduce computational burden.

15.2 Robot Base to Static Obstacles

$$h_{\text{base-static}} = \|\mathbf{b}_i - \mathbf{c}_{\text{obs}}\|^2 - (r_{\text{obs}} + d_{\text{min,base-obs}})^2$$
$$\dot{h}_{\text{base-static}} = 2(\mathbf{b}_i - \mathbf{c}_{\text{obs}})^T\mathbf{v}_i$$

15.3 End-Effector to Static Obstacles

$$h_{\text{ee-static}} = \|\mathbf{e}_i - \mathbf{c}_{\text{obs}}\|^2 - (r_{\text{obs}} + d_{\text{min,obj-obs}})^2$$
$$\dot{h}_{\text{ee-static}} = 2(\mathbf{e}_i - \mathbf{c}_{\text{obs}})^T\mathbf{v}_e$$

where $\mathbf{v}_e$ is computed using the Jacobian:

$$\mathbf{v}_e = \mathbf{v}_b + \mathbf{J}_{\text{arm}}\dot{\mathbf{a}}$$

15.4 Dynamic Obstacle Constraints

For dynamic obstacles, the relative velocity accounts for obstacle motion:

$$h_{\text{dyn}} = \|\mathbf{b}_i - \mathbf{c}_{\text{dyn}}\|^2 - (r_{\text{dyn}} + d_{\text{min}})^2$$

$$\dot{h}_{\text{dyn}} = 2(\mathbf{b}_i - \mathbf{c}_{\text{dyn}})^T(\mathbf{v}_i - \mathbf{v}_{\text{dyn}})$$

where $\mathbf{v}_{\text{dyn}}$ is the obstacle velocity computed from trajectory prediction.

15.5 Angular Non-Collinearity

This constraint ensures the robot's grasp remains geometrically valid, preventing the manipulation triangle from becoming collinear with pentagon edges. The barrier function computes angles and ensures minimum angular separation with gain $\gamma_{\text{angular}} = 15.0$.

16 Rigidity Constraints

To maintain a rigid pentagon formation, the code constrains all pairwise distances between robot end-effectors and bases (Lines 532–576). For $n_r = 3$ robots, there are
$binom32 = 3$ end-effector pairs and 3 base pairs.

16.1 End-Effector Rigidity

For each pair (i, j):

$$d_{ij,0}(1 - \varepsilon_{\text{rig}}) \leq \|\mathbf{e}_i - \mathbf{e}_j\| \leq d_{ij,0}(1 + \varepsilon_{\text{rig}})$$

with tolerance $\varepsilon_{\text{rig}} = 0.008$ (0.8%), where $d_{ij,0}$ is the initial pairwise distance.

16.2 Base Rigidity

Similarly for base positions:

$$d_{ij,\text{base},0}(1 - \varepsilon_{\text{base}}) \leq \|\mathbf{b}_i - \mathbf{b}_j\| \leq d_{ij,\text{base},0}(1 + \varepsilon_{\text{base}})$$

with relaxed tolerance $\varepsilon_{\text{base}} = 0.03$ (3%). The looser base tolerance allows more maneuverability while still preventing outward swings.

These constraints prevent the formation from deforming, ensuring the grasp remains valid and the pentagon object maintains its structural integrity throughout motion.

17 Solver Configuration

The code configures IPOPT (Interior Point OPTimizer) with specific parameters (Lines 579–591):

- **Print Level**: 0 (minimal console output)
- **Warm Start**: `yes` (initializes from previous solution)
- **Maximum Iterations**: 3000 (sufficient for complex problems)
- **Tolerance**: 10^{-6} (high accuracy)
- **Acceptable Tolerance**: 10^{-5} (fallback accuracy)
- **Acceptable Iterations**: 20 (iterations before accepting suboptimal solution)

These settings balance solution quality with computational speed. Warm-starting from the previous MPC solution is critical for achieving real-time performance, where each iteration must solve in under 0.1 seconds.

18 Simulation Loop

The main simulation (Lines 594–690) runs for 40 timesteps, implementing a receding-horizon control strategy:

1. **Reference Extraction**: Extract the reference trajectory window covering the next N steps

2. **Parameter Update**: Set current state and obstacle positions as optimization parameters
3. **Warm-Start Initialization**: Initialize decision variables from previous solution:

$$\mathbf{X}^{init} = [\mathbf{X}^{prev}(\cdot, 2:N{+}1), \mathbf{X}^{prev}(\cdot, N{+}1)], \quad \mathbf{U}^{init} = [\mathbf{U}^{prev}(\cdot, 2:N), \mathbf{U}^{prev}(\cdot, N)]$$

4. **Optimization**: Solve the NMPC problem using IPOPT
5. **Control Application**: Apply the first control input:

$$\mathbf{x}(k+1) = \mathbf{x}(k) + \Delta t \cdot \mathbf{u}(k)$$

6. **Logging**: Record solve time, status, and constraint violations

When solving fails, the code attempts recovery by extracting the debug solution, which often provides a feasible (though suboptimal) control despite convergence issues. State bounds are enforced via clipping after control application.

19 Animation Generation

The visualization code (Lines 693–808) creates a detailed real-time animation showing the system evolution:

19.1 Rendered Elements

- **Static Obstacles**: Hexagon-shaped regions colored in red
- **Dynamic Obstacles**: Orange hexagons with motion trails showing past positions
- **Robots**: Circular bases colored distinctly for each robot
- **Robot Arms**: Lines showing the two-link kinematic chain
- **Robot Footprints**: Triangular regions representing the contact/grasp area
- **Pentagon Object**: Black polygon connecting the five end-effectors
- **Reference Trajectory**: Dashed black line showing the desired object path

19.2 Computational Steps

For each animation frame:

1. Compute forward kinematics for all end-effectors
2. Update robot base positions and arm line plots
3. Calculate triangular footprints using the tangency calculation

4. Render the pentagon object as a polygon through the five end-effector points
5. Update dynamic obstacle positions from trajectory data
6. Display current time and step information

The animation is saved as an MP4 video at 20 fps using FFMpegWriter, creating a clear visual record of the system's behavior.

20 Key Innovations and Technical Contributions

This implementation combines several advanced control techniques:

20.1 Velocity-Based CBFs

Instead of traditional position-based barrier functions, this code uses velocity-based CBFs that incorporate state derivatives. This allows the system to predict collisions and react proactively, rather than only responding to imminent threats. The gain γ controls the exponential convergence rate: larger γ ensures faster constraint satisfaction.

20.2 Rigid Formation Control

The all-pairwise distance constraints maintain the pentagon formation's geometric integrity. By constraining both end-effector and base distances, the system prevents internal deformation while allowing coordinated motion. The relaxed base tolerance enables maneuverability in constrained spaces.

20.3 Dynamic Obstacle Handling

Incorporating obstacle velocity predictions into the CBF formulation enables safety-aware navigation in dynamic environments. The system doesn't just react to current obstacle positions but predicts future collisions based on velocity.

20.4 Cooperative Manipulation

Multiple robots working together to transport an object while maintaining formation represents a complex multi-objective control problem. The formation cost term ensures the object remains rigid while tracking references, and CBF constraints prevent inter-robot collisions.

20.5 Real-Time MPC via Warm-Starting

The receding-horizon optimization with warm-starting from the previous solution achieves real-time performance. Each solve builds on the previous one, significantly reducing computation time compared to solving from scratch. This enables deployment in time-critical applications.

20.6 Jacobian-Based Velocity Computation

The code explicitly computes end-effector velocities using the geometric Jacobian:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial \mathbf{e}}{\partial q_1} & \frac{\partial \mathbf{e}}{\partial d_2} & \frac{\partial \mathbf{e}}{\partial q_3} \end{bmatrix}$$

This enables proper velocity-based constraints for the manipulator's end-effector, crucial for safe interaction with obstacles and maintaining grasp validity.

21 Conclusion

This NMPC implementation demonstrates sophisticated multi-agent coordination combining trajectory tracking, formation maintenance, manipulation, and obstacle avoidance. The integration of velocity-based control barrier functions, rigid formation constraints, and cooperative manipulation creates a system capable of complex autonomous behaviors. The use of CasADi for symbolic computation and IPOPT for non-linear optimization provides a powerful framework for solving this challenging control problem in near-real-time.

The code serves as a comprehensive example of modern control theory applied to multi-robot systems, with practical applications in warehouse automation, collaborative manufacturing, and autonomous transportation of objects in cluttered environments.